# OPTIMIZING TOKEN USAGE WITH RAG KNOWLEDGE ASSISTANT

Technical Investigation and Experimental Evaluation

## A. Identify Optimization Techniques

### 1.1 Retrieval Optimization (RAG)

With the RAG methodology, we retrieve only relevant documents or passages. To achieve this, we first divide the documents into smaller chunks. Based on the user's query, we then analyze these chunks using similarity filtering and relevance thresholds. Afterwards, the most relevant chunks are selected based on their similarity scores.

This technique primarily reduces **input token usage**.

**Expected Benefits:** Smaller context sent to the LLM, lower API costs, and faster response generation.

**Potential Risks:** Missing important information, lower answer accuracy if relevant chunks are excluded, and dependency on retrieval quality.

### 1.2 Conversation History Summarization

This methodology replaces older conversation messages with compact summaries while keeping recent messages in full. It preserves key facts, decisions, and user preferences. As the conversation grows, the model updates these summaries to maintain the essential context.

This technique primarily reduces **input token usage**.

**Expected Benefits:** Reduced context window usage and lower costs for long conversations.

**Potential Risks:** Loss of important details, summarization errors, and accumulation of errors over multiple summaries.

### 1.3 Prompt Optimization

This methodology shortens system instructions and removes redundant instructions and examples. It uses concise and structured prompts to avoid repeating information across different prompt components while preserving essential instructions.

This technique primarily reduces **input token usage**.

**Expected Benefits:** Lower token usage per request and simple implementation. It also does not require major changes to the model architecture.

**Potential Risks:** Loss of important instructions, reduced consistency in responses, and a potential decrease in reliability.

### 1.4 Context Compression

This methodology reduces the amount of retrieved information before passing it to the LLM. It divides retrieved documents into smaller parts and selects the most relevant information based on the user's query. Less relevant information is removed while trying to preserve important details.

This technique primarily reduces **input token usage**.

**Expected Benefits:** Smaller input context, lower token consumption, and reduced generation costs.

**Potential Risks:** Loss of important information, incomplete context, and reduced answer accuracy if necessary details are removed.

## B. Selected Approach

### Retrieval Optimization with Context Compression

In this method, I focused on retrieving relevant documents instead of the entire knowledge base. Markdown documents are divided into smaller chunks based on their sections. Embeddings are generated using `nomic-embed-text`. To match the user's query with the document chunks, cosine similarity is used. The top five most relevant chunks are selected, reducing irrelevant information in the LLM context.

On top of that, the retrieved chunks are divided into smaller paragraphs and ranked based on their relevance to the user's query. The most relevant paragraphs are selected within an approximate 200-token budget. This removes lower-priority information before passing the context to the LLM.

I selected these techniques because retrieved context is one of the main sources of input token consumption in RAG systems. By combining retrieval optimization with context compression, token usage can be reduced while keeping the relevant information needed for answer generation. These techniques are also relatively simple to implement and compare with a baseline approach.

To implement this, a Python-based RAG prototype was created using Ollama for local LLM execution. I used `qwen3:4b-instruct` for answer generation and `nomic-embed-text` for document and query embeddings. For document processing, I implemented section-aware chunking and Top-K semantic retrieval with K = 5.

To compare the effect of these modifications, I created baseline and optimized versions. The baseline retrieves five chunks without additional

compression. However, the optimized version uses the same five retrieved chunks with paragraph-level compression. To test them, identical questions and answer instructions are used for both versions. Token consumption, response time, and answer quality are then compared.

For evaluation, input and output token counts are measured along with response time. In addition, the RAGAS evaluation framework is used to assess factual correctness, faithfulness, context precision, and context recall. These metrics help determine whether reducing token usage negatively affects the quality and reliability of the generated answers.

## C. Experimental Proposal and Results

The experiment compares the baseline RAG agent with the optimized RAG agent to evaluate token reduction and its impact on answer quality.

For the experiment, I prepared 10 test questions with predefined reference answers based on laboratory operational knowledge. The questions cover equipment maintenance, laboratory access, data security, and safety procedures. The dataset includes simple factual questions, more complex questions requiring multiple pieces of information, and one question for which the information is unavailable.

Both agents use the same documents, retrieval method, language model, and question instructions. The baseline agent passes the five retrieved chunks directly to the LLM, while the optimized agent applies paragraph-level context compression before generating the answer.

For each question, input tokens, output tokens, total token consumption, and response time are measured. RAGAS is used to evaluate answer quality by comparing the generated answers with reference answers and retrieved contexts.

### Experimental Results

| Metric (Average) | Baseline | Optimized |
|---|---|---|
| Input Tokens | 342.5 | 251.8 |
| Output Tokens | 29.0 | 33.3 |
| Total Tokens | 371.5 | 285.1 |
| Response Time (s) | 1.30 | 1.51 |
| Factual Correctness | 0.396 | 0.423 |
| Faithfulness | 0.889 | 0.949 |
| Context Precision | 0.849 | 0.683 |
| Context Recall | 0.889 | 0.889 |

The experimental results show that the optimized agent reduced average input token consumption by approximately **26.5%** and total token consumption by **23.3%**. However, the average response time increased from 1.30 to 1.51 seconds due to the additional compression process.

In terms of answer quality, factual correctness slightly improved from 0.396 to 0.423, while faithfulness increased from 0.889 to 0.949. Context recall remained unchanged at 0.889. However, context precision decreased from 0.849 to 0.683, indicating that compression did not improve all evaluation metrics.

Both agents also correctly responded with "I don't know" when the required information was unavailable.

### Conclusion

Based on the experiment, context compression successfully reduced token consumption while maintaining or slightly improving some answer quality metrics. However, the decrease in context precision and the increase in response time show that there are trade-offs between token efficiency, answer quality, and performance.

Therefore, further testing with a larger dataset and different compression limits would help identify an optimal balance between token reduction and answer reliability.

## D. References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. https://arxiv.org/abs/2005.11401

[2] H. Jiang et al., "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," *Proceedings of EMNLP*, pp. 13358–13376, 2023. https://aclanthology.org/2023.emnlp-main.825/

[3] Ollama, "Ollama API Documentation." https://docs.ollama.com/api

[4] Nomic AI, "Nomic Embed Text: Text Embedding Model." https://ollama.com/library/nomic-embed-text

[5] Ragas, "Ragas Documentation: Evaluation Metrics for Retrieval-Augmented Generation." https://docs.ragas.io/en/latest/concepts/metrics/available_metrics/

[6] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation," *International Conference on Learning Representations (ICLR)*, 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/bda88ed2892f5e61c9a9bf215c566913-Abstract-Conference.html